

YAML

openapi: 3.0.3

info:

title: Task Tracker API

version: 1.0.0

description: API for creating and retrieving tasks.

paths:

/api/v1/tasks:

post:

summary: Create a new task

operationId: createTask

requestBody:

required: true

content:

application/json:

schema:

\$ref: '#/components/schemas/CreateTaskRequest'

responses:

'200':

description: Task created successfully

content:

application/json:

schema:

\$ref: '#/components/schemas/Task'

'400':

description: Invalid request

content:

application/json:

schema:

\$ref: '#/components/schemas/ErrorResponse'

/api/v1/tasks/{id}:

get:

summary: Get a task by ID

operationId: getTaskById

parameters:

- name: id

in: path

required: true

description: Unique identifier of the task

schema:

type: integer

example: 42

responses:

'200':

description: Task retrieved successfully
content:
 application/json:
 schema:
 \$ref: '#/components/schemas/Task'
'400':
 description: Invalid task ID
 content:
 application/json:
 schema:
 \$ref: '#/components/schemas/ErrorResponse'

components:
 schemas:
 CreateTaskRequest:
 type: object
 required:
 - title
 - description
 - due_date
 properties:
 title:
 type: string
 description: Title of the task
 example: Complete API documentation
 description:
 type: string
 description: Detailed description of the task
 example: Write and review the Swagger documentation.
 due_date:
 type: string
 format: date-time
 description: Due date and time for the task
 example: '2026-09-25T17:00:00Z'

Task:
 type: object
 properties:
 id:
 type: integer
 example: 42
 user_id:
 type: integer
 example: 7
 title:
 type: string
 example: Complete API documentation
 description:

type: string
example: Write and review the Swagger documentation.
status:
type: string
example: pending
due_date:
type: string
format: date-time
example: '2026-09-25T17:00:00Z'
created_at:
type: string
format: date-time
example: '2026-09-20T18:30:00Z'

ErrorResponse:
type: object
properties:
error:
type: string
example: Invalid request

Jest/Supertest Task API Unit Tests

```
const request = require('supertest');  
const app = require('../app');  
  
describe('Task API', () => {  
  describe('POST /api/v1/tasks', () => {  
    test('Test Case 1: Successful task creation', async () => {  
      const task = {  
        title: 'Complete API testing',  
        description: 'Write automated tests for the Task Tracker API',  
        due_date: '2026-09-25T17:00:00Z'  
      };  
  
      const response = await request(app)  
        .post('/api/v1/tasks')  
        .send(task)  
        .set('Accept', 'application/json');  
  
      expect(response.status).toBe(201);  
      expect(response.body).toHaveProperty('id');  
      expect(response.body.title).toBe(task.title);
```

```
});  
});  
  
describe('GET /api/v1/tasks/:id', () => {  
  test('Test Case 2: Getting task with invalid ID', async () => {  
    const response = await request(app)  
      .get('/api/v1/tasks/999999')  
      .set('Accept', 'application/json');  
  
    expect(response.status).toBe(404);  
  });  
});  
});
```

Run with:

```
npm install --save-dev jest supertest  
npx jest
```

These tests intentionally check **201** for creation and **404** for a nonexistent task ID, matching the requested QA scenarios.